

Parallel Traffic-Light Optimization on an HPC Cluster

Large Scale Computing — Project Report

Grzegorz Mróz and Katarzyna Kucharzyk

June 12, 2026

Abstract

This project uses traffic-light timing optimization as a realistic, embarrassingly parallel workload to demonstrate core High Performance Computing (HPC) concepts: SLURM job arrays, parallel scaling, search-strategy comparison, parameter sensitivity analysis, and an idealized Monte Carlo baseline. Every candidate traffic-light configuration is evaluated by an independent simulation, which makes the workload trivially parallelizable across the nodes of the PLGrid/Ares cluster. We report four experiments. The scaling study confirms near-linear speedup (efficiency $\geq 91\%$ up to 100 workers). An evolutionary algorithm (EA) outperforms random and grid search at an equal simulation budget, reducing the mean objective by roughly $6\times$ and cutting average vehicle waiting time by 10.7% relative to a hand-crafted fixed-time baseline. A sensitivity sweep shows that, for a fixed budget, more generations beat larger populations. Finally, a Monte Carlo π -estimator on 5×10^7 samples illustrates perfect parallelism with zero inter-task communication.

1 Introduction

Optimizing the timing of traffic lights is computationally expensive because each candidate configuration must be evaluated in a traffic simulation. Since the configurations are mutually independent, the search is an *embarrassingly parallel* problem: thousands of simulations can run simultaneously with no communication between them. This makes traffic-light optimization an ideal vehicle for studying HPC batch processing.

The goal of this project is **not** to build a production traffic controller, but to use a scalable prototype to demonstrate and measure HPC properties: how the workload scales with the number of workers, how different optimization strategies trade off quality against compute budget, and how an idealized parallel workload behaves as a reference point.

2 Methodology

2.1 Traffic-light model and objective

Each intersection is controlled by a fixed-time signal program described by a JSON configuration with the following parameters: `cycle_length`, North–South green time (`green_ns`), East–West green time (`green_ew`), `yellow_time`, `all_red_time`, and a coordination `offset`. All configurations satisfy the timing constraint

$$\text{green_ns} + \text{green_ew} + 2 \cdot \text{yellow_time} + 2 \cdot \text{all_red_time} \leq \text{cycle_length}.$$

The optimization objective is to **minimize the average vehicle waiting time** (`avg_waiting_time`, in seconds) over a one-hour simulated horizon on a 3×3 grid of intersections under medium traffic demand.

2.2 Simulation backends

The architecture follows one strict rule: *nothing outside the simulator module knows how a backend works internally*. Every backend consumes one JSON config and emits exactly one CSV row with a fixed metric schema. Three backends are provided:

- **python** — a discrete-time queueing model of the intersections. Fast (~ 7 ms per simulation) and dependency-free.
- **sumo** — a microscopic simulation using the SUMO traffic simulator (~ 1.4 s per simulation). Used here for visualization.
- **mc** — a Monte Carlo π -estimator used as an idealized embarrassingly-parallel reference.

Because all backends share the same input/output contract, the SLURM workflow, aggregation, and analysis code are backend-agnostic. The four headline experiments were executed on Ares with the **python** backend (SUMO was not available in the cluster module environment); the SUMO backend was used locally to render the traffic animation.

2.3 HPC execution model

Each simulation is one task of a SLURM *job array*. Concurrency is controlled with the array throttle (`-array=0-P%N`), which limits how many of the P tasks run simultaneously to N . Results are written as independent CSV files and aggregated afterwards, so there is no shared state and no synchronization during evaluation. The EA introduces the only synchronization point: a barrier between generations, where the next population is bred from the evaluated parents.

3 Experiment 1: Parallel Scaling

Setup. A fixed workload of $P = 100$ configurations is evaluated while varying the number of concurrent workers $N \in \{1, 2, 5, 10, 20, 50, 100\}$ (strong scaling). Each task records its own runtime. The wall-clock time for N workers is modelled as the makespan of scheduling the measured per-task runtimes onto N workers (longest-processing-time greedy schedule); the sequential baseline $T(1)$ is the sum of all task runtimes. Speedup is $S(N) = T(1)/T(N)$ and efficiency is $E(N) = S(N)/N$.

Table 1: Strong-scaling results ($P = 100$ tasks).

Workers N	Wall time $T(N)$ [s]	Speedup $S(N)$	Efficiency $E(N)$
1	0.6875	1.00	100.0%
2	0.3438	2.00	100.0%
5	0.1376	5.00	99.9%
10	0.0688	9.99	99.9%
20	0.0345	19.93	99.6%
50	0.0139	49.46	98.9%
100	0.0075	91.67	91.7%

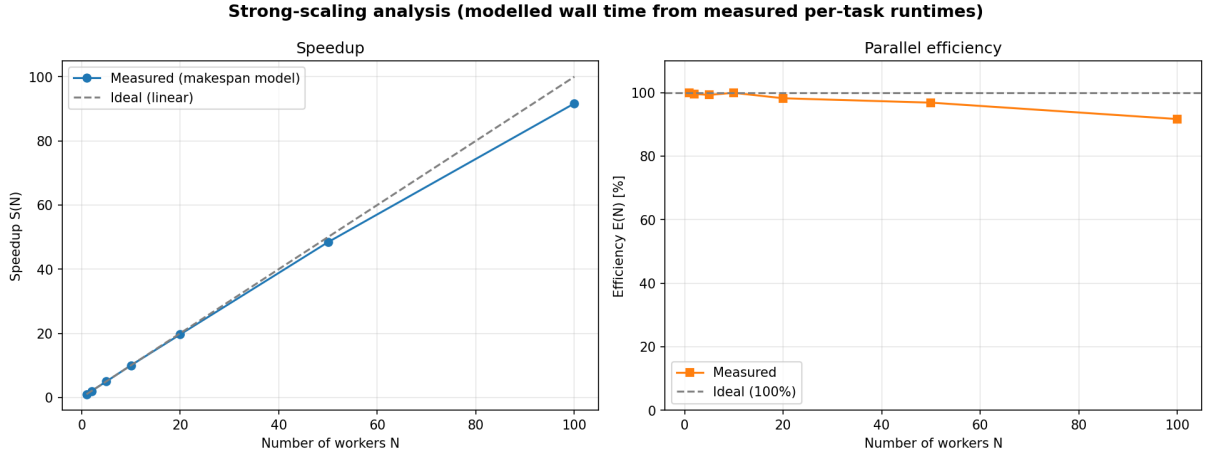


Figure 1: Strong scaling. Left: measured speedup tracks the ideal linear curve almost exactly. Right: parallel efficiency stays above 98% up to $N = 50$ and only drops to 91.7% at $N = 100$, where granularity (one task per worker) makes load imbalance unavoidable.

Discussion. The workload exhibits essentially ideal strong scaling, as expected for an embarrassingly parallel problem: there is no inter-task communication, so the only loss of efficiency comes from load granularity. At $N = 100$ each worker handles a single task, so the longest task dictates the makespan and efficiency falls to 91.7% — a textbook illustration of why the number of tasks should comfortably exceed the number of workers.

4 Experiment 2: EA vs. Random vs. Grid Search

Setup. Three search strategies are compared: *random search* (300 random configs), *grid search* (a regular lattice over cycle length and green splits, 72 points — the natural resolution of the grid before combinatorial blow-up), and an *evolutionary algorithm* (population of 50 over 6 generations, i.e. 300 evaluations). Random search and the EA thus operate on an equal budget of 300 evaluations, while grid search samples the space more sparsely. Quality is the best/mean/worst `avg_waiting_time`. As an absolute reference we also report a hand-crafted *fixed-time baseline* (60s cycle, 30/25 green split, zero offset), evaluated on the same backend and demand.

Table 2: Search-strategy comparison (avg. waiting time, seconds; lower is better). EA statistics describe the final-generation population of 50 individuals; its total budget is 300 evaluations.

Strategy	Evaluations	Best	Mean	Worst
Fixed-time baseline	1	12.25	—	—
Random search	300	11.03	212.87	1070.36
Grid search	72	11.55	138.40	710.20
Evolutionary algorithm	300 (50 × 6 gen.)	10.94	33.94	366.89

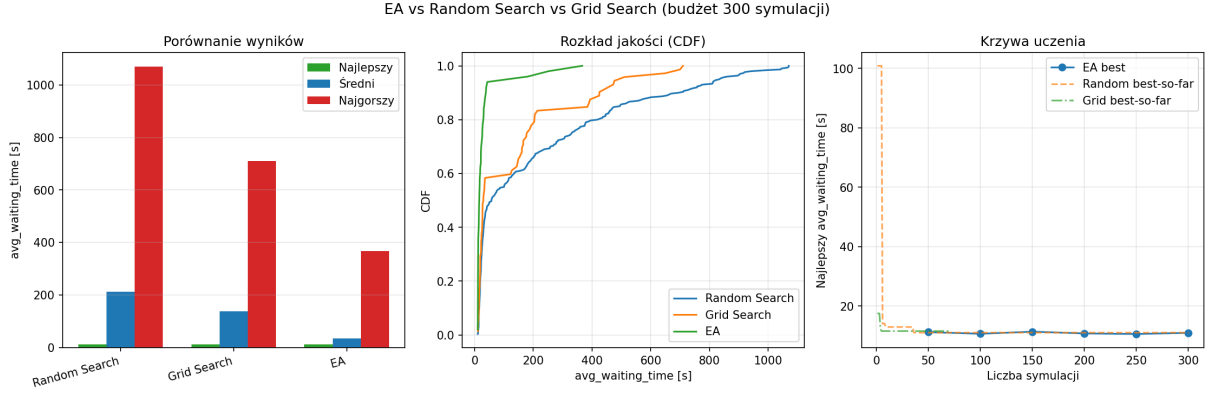


Figure 2: Left: best/mean/worst objective per strategy. Middle: cumulative distribution of solution quality — the EA curve rises fastest, i.e. it produces far fewer poor solutions. Right: best-so-far learning curve against the number of simulations.

Discussion. All three strategies find a similar *best* solution (≈ 11 s), because good configurations are not especially rare. The decisive difference is the *distribution*: the EA’s mean objective (33.9 s) is about $6\times$ lower than random search (212.9 s) and $4\times$ lower than grid search, and its worst case is roughly $3\times$ better. The CDF makes this explicit — the EA concentrates its budget in the high-quality region instead of wasting evaluations on poor candidates. For a fixed compute budget, the EA therefore extracts substantially more value, which is the central motivation for using guided search on HPC. Grid search received fewer evaluations than the other two methods, but this only strengthens the conclusion for random search (equal budget, $6\times$ worse mean) and does not change the ranking: the EA dominates on every reported statistic.

Practical payoff. Measured against the fixed-time baseline (12.25 s average waiting under identical demand — 3262 vs. 3268 vehicle arrivals), the best evolved configuration achieves 10.94 s, i.e. a **10.7% reduction in average vehicle waiting time**. This is the bottom line of the optimization: a sensibly hand-tuned signal plan is already decent, yet a few CPU-minutes of parallel guided search buy a double-digit relative improvement.

5 Experiment 3: EA Parameter Sensitivity

Setup. The EA budget is fixed at 300 evaluations and split between population size and number of generations in seven ways, from 300×1 (pure random) to 15×20 (deep, narrow search).

Table 3: EA sensitivity at a fixed budget of 300 evaluations.

Population	Generations	Best avg. waiting time [s]
300	1	11.03
150	2	11.45
100	3	11.47
60	5	10.88
50	6	11.51
30	10	10.61
15	20	10.55

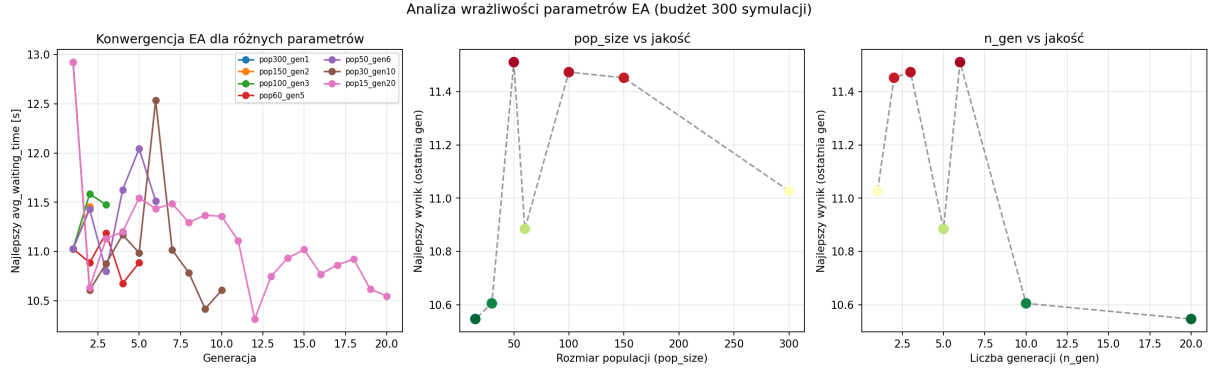


Figure 3: Left: best-fitness convergence per configuration. Middle and right: final best objective as a function of population size and of the number of generations.

Discussion. At a fixed budget there is a clear exploration vs. exploitation trade-off. Configurations with a single generation (300×1 , equivalent to random search) and very large populations leave quality on the table, whereas allocating the budget to *more generations* (15×20 , 30×10) yields the best results. This indicates that iterative selection pressure — not raw population diversity — drives improvement for this problem. The differences are modest (all within ≈ 1 s), reflecting that the EA used here has no elitism, so the incumbent best can be lost between generations; adding elitism is the natural next improvement.

6 Experiment 4: Monte Carlo π — An Idealized Parallel Baseline

Setup. As a reference for “perfect” parallelism, 500 independent tasks each draw 10^5 random points in the unit square and count those inside the quarter circle, giving $\pi \approx 4$ (hits/samples). The combined estimate uses 5×10^7 samples. Unlike the EA, there is no synchronization barrier whatsoever.

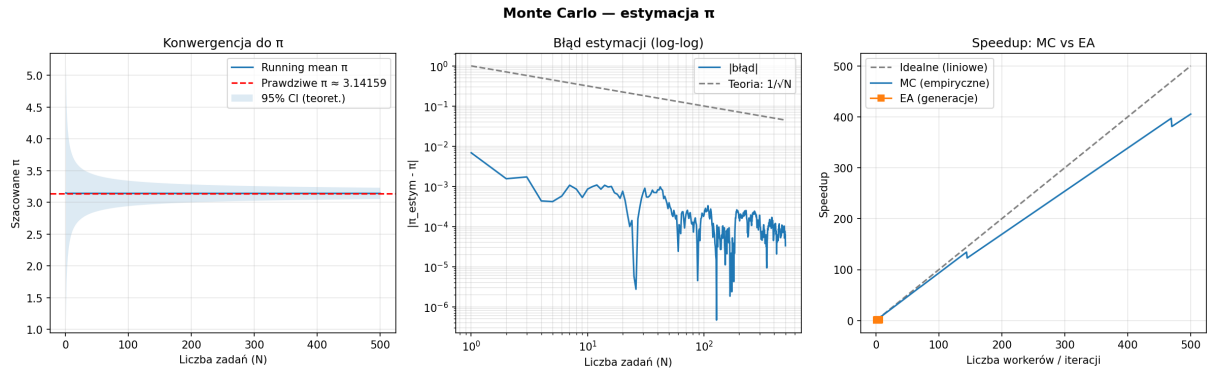


Figure 4: Left: running estimate of π converging to the true value within its theoretical 95% confidence band. Middle: absolute error vs. number of tasks on log-log axes, following the expected $1/\sqrt{N}$ trend. Right: speedup — Monte Carlo is linear (no communication) while the EA’s per-generation barrier limits its scaling.

Results. The aggregate estimate is $\pi \approx 3.141626$ versus the true 3.141593, an absolute error of 3.3×10^{-5} , consistent with the $1/\sqrt{N}$ law. This experiment isolates the ideal case: speedup is perfectly linear because tasks never communicate, in contrast to the EA, whose generation barrier introduces a (small) serial fraction and caps achievable speedup — a concrete illustration of Amdahl’s law.

7 Visualization

The SUMO backend produces per-vehicle traces that are rendered into an animation of the optimized 3×3 grid (`results/plots/anim_best.gif`). Figure 5 shows a representative frame: cyan markers indicate active green phases and red markers show queued vehicles waiting at red lights.

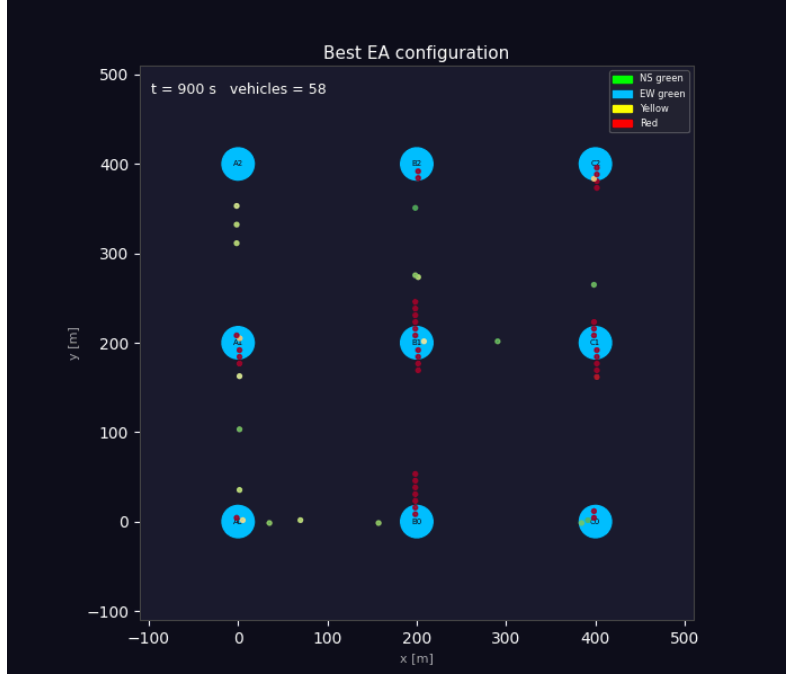


Figure 5: Snapshot of the SUMO microsimulation for the best configuration ($t = 900\text{s}$, 58 vehicles in the network).

8 Conclusions

- **Embarrassingly parallel, near-ideal scaling.** The simulation workload scales almost linearly, with efficiency above 98% up to 50 workers; the only loss comes from task granularity at $N = 100$.
- **Guided search wins at equal cost.** The EA delivers a $\sim 6\times$ lower mean objective and a markedly better worst case than random or grid search for the same simulation budget.
- **Optimization pays off in traffic terms.** The best evolved signal plan reduces average vehicle waiting time by 10.7% relative to a hand-crafted fixed-time baseline ($12.25 \rightarrow 10.94\text{s}$).
- **Spend budget on iterations.** Under a fixed budget, more generations outperform larger populations, highlighting the value of selection pressure.
- **Synchronization has a cost.** The communication-free Monte Carlo baseline scales perfectly, whereas the EA’s generation barrier introduces a serial fraction — a practical demonstration of Amdahl’s law.

Overall, the project shows how a realistic optimization problem maps cleanly onto HPC batch infrastructure and how the choice of algorithm, not just raw compute, determines the value obtained from a fixed allocation of CPU hours.